

```

from dataclasses import dataclass
from typing import Dict, List, Tuple
import math

@dataclass
class Option:
    name: str
    revenue_upside: float
    implementation_cost: float
    strategic_fit: float
    execution_risk: float
    time_to_impact: float

class StrategicDecisionAI:
    def __init__(self, weights: Dict[str, float]):
        self.weights = weights

    def normalize(self, values: List[float], reverse: bool = False) -> List[float]:
        lo, hi = min(values), max(values)
        if math.isclose(lo, hi):
            return [1.0 for _ in values]
        scaled = [(v - lo) / (hi - lo) for v in values]
        return [1 - x for x in scaled] if reverse else scaled

    def score_options(self, options: List[Option]) -> List[Tuple[Option, float, Dict[str, float]]]:
        rev = self.normalize([o.revenue_upside for o in options])
        cost = self.normalize([o.implementation_cost for o in options], reverse=True)
        fit = self.normalize([o.strategic_fit for o in options])
        risk = self.normalize([o.execution_risk for o in options], reverse=True)
        tti = self.normalize([o.time_to_impact for o in options], reverse=True)

        ranked = []
        for i, o in enumerate(options):
            factors = {
                'revenue_upside': rev[i],
                'implementation_cost': cost[i],
                'strategic_fit': fit[i],
                'execution_risk': risk[i],
                'time_to_impact': tti[i],
            }
            score = sum(self.weights[k] * v for k, v in factors.items())
            ranked.append((o, score, factors))
        return sorted(ranked, key=lambda x: x[1], reverse=True)

    def hierarchical_decision(self, options: List[Option]) -> Dict:
        screened = [o for o in options if o.execution_risk <= 0.70 and o.strategic_fit >= 0.50]
        ranked = self.score_options(screened)
        portfolio = {
            'primary_bet': ranked[0][0].name if ranked else None,
            'secondary_bet': ranked[1][0].name if len(ranked) > 1 else None,
            'avoid': [o.name for o in options if o not in screened],
            'ranking': [
                {
                    'name': o.name,
                    'score': round(score, 3),
                    'factors': {k: round(v, 3) for k, v in factors.items()}
                }
                for o, score, factors in ranked
            ]
        }
        return portfolio

if __name__ == '__main__':
    options = [
        Option('Expand to Nordics', 8.5, 4.0, 0.92, 0.35, 6),
        Option('Launch AI Copilot', 9.7, 6.5, 0.88, 0.55, 4),
        Option('Acquire Small Rival', 7.9, 9.5, 0.73, 0.68, 10),
        Option('Cut Prices Aggressively', 6.2, 3.0, 0.42, 0.72, 2),
    ]

    weights = {
        'revenue_upside': 0.30,

```

```
        'implementation_cost': 0.15,  
        'strategic_fit': 0.25,  
        'execution_risk': 0.20,  
        'time_to_impact': 0.10,  
    }  
  
    ai = StrategicDecisionAI(weights)  
    result = ai.hierarchical_decision(options)  
    print(result)
```